Chapter 3 - Python Function

October 11, 2020

---

1 Python Functions - What is a Function?

Let's imagine that we are creating a program that greets customers as they enter a grocery store. We want a big screen at the entrance of the store to say:

```
Welcome to Engrossing Grocers.
Our special is mandarin oranges.
Have fun shopping!
```

We have learned to use print statements for this purpose:

```
print("Welcome to Engrossing Grocers.")
print("Our special is mandarin oranges.")
print("Have fun shopping!")
```

Every time a customer enters, we call these three lines of code. Even if only 3 or 4 customers come in, that's a lot of lines of code required.

In Python, we can make this process easier by assigning these lines of code to a function. We'll name this function greet_customer. In order to call a function, we use the syntax function_name(). The parentheses are important! They make the code inside the function run. In this example, the function call looks like:

```
greet_customer()
```

Every time we call greet_customer(), we would see:

```
Welcome to Engrossing Grocer's.
Our special is mandarin oranges.
Have fun shopping!
```

Having this functionality inside **greet_customer()** is better form, because we have isolated this behavior from the rest of our code. Once we determine that * **greet_customer()** works the way we want, we can reuse it anywhere and be confident that it greets, without having to look at the implementation. We can get the same output, with less repeated code. Repeated code is generally more error prone and harder to understand, so it's a good goal to reduce the amount of it.

A function is a block of code which only runs when it is called.

You can pass data, known as parameters, into a function.

A function can return data as a result. ## Creating a Function

In Python a function is defined using the **def** keyword:

1.1 Write a Function

We have seen the value of simple functions for modularizing code. Now we need to understand how to write a function. To write a function, you must have a heading and an indented block of code. The heading starts with the keyword def and the name of the function, followed by parentheses, and a colon. The indented block of code performs some sort of operation. This syntax looks like: `def function_name():  some code` For our greet_customer() example, the function definition looks like:

```
def greet_customer():
  print("Welcome to Engrossing Grocers.")
  print("Our special is mandarin oranges.")
  print("Have fun shopping!")
```

And somewhere onside code we can call function like this way:

```
greet_customer()
# prints greeting lines
```

The keyword **def** tells Python that we are defining a function. This function is called greet_customer. Everything that is indented after the : is what is run when greet_customer() is called. So every time we call greet_customer(), the three print statements run.

1.2 Calling a Function

To call a function, use the function name followed by parenthesis:

[1]:
```
#Example function call and remember be carefully indentation for all statements␣
↪that belong to the performance of the function
def my_function():
  print("Hello from a function")
  print("Myös minut pitää tulostaa")


my_function()
```

```
Hello from a function
Myös minut pitää tulostaa
```

1.3 Arguments

Information can be passed into functions as arguments.

Arguments are specified after the function name, inside the parentheses. You can add as many arguments as you want, just separate them with a comma.

The following example has a function with one argument (fname). When the function is called, we pass along a first name, which is used inside the function to print the full name:

[2]:
```
def my_function(fname):
  print(fname + " Refsnes")

my_function("Emil")
my_function("Tobias")
my_function("Linus")
```

```
Emil Refsnes
Tobias Refsnes
Linus Refsnes
```

1.4 Number of Arguments

By default, a function must be called with the correct number of arguments. Meaning that if your function expects 2 arguments, you have to call the function with 2 arguments, not more, and not less.

[3]:
```
# This function expects 2 arguments, and gets 2 arguments:
def my_function(fname, lname):
  print(fname + " " + lname)

my_function("Emil", "Refsnes")
```

```
Emil Refsnes
```

Note!! If you try to call the my_function with 1 or 3 arguments, you will get an error.

[4]:
```
# This function expects 2 arguments, and gets 2 arguments:
def my_function(fname, lname):
  print(fname + " " + lname)

my_function("Emil")
```

```
␣
↪---------------------------------------------------------------------------

       TypeError                                 Traceback (most recent call␣
↪last)
```

```
    <ipython-input-4-e64cf40dfe2a> in <module>
      3   print(fname + " " + lname)
      4
----> 5 my_function("Emil")


    TypeError: my_function() missing 1 required positional argument: 'lname'
```

1.5 Arbitrary Arguments, *args

If you do not know how many arguments that will be passed into your function, add a * before the parameter name in the function definition.

This way the function will receive a tuple of arguments, and can access the items accordingly:

[5]:
```
# If the number of arguments is unknown, add a * before the parameter name:
def my_function(*kids):
  print("The youngest child is " + kids[2])

my_function("Emil", "Tobias", "Linus")
```

```
The youngest child is Linus
```

Arbitrary Arguments are often shortened to *args in Python documentations.

1.6 Keyword Arguments

You can also send arguments with the key = value syntax.

This way the order of the arguments does not matter.

[6]:
```
def my_function(child3, child2, child1):
  print("The youngest child is " + child3)

my_function(child1 = "Emil", child2 = "Tobias", child3 = "Linus")
```

```
The youngest child is Linus
```

1.7 Passing a List as an Argument

You can send any data types of argument to a function (string, number, list, dictionary etc.), and it will be treated as the same data type inside the function.

E.g. if you send a List as an argument, it will still be a List when it reaches the function:

[7]:
```
def my_function(food):
  for x in food:
    print(x)

fruits = ["apple", "banana", "cherry"]

my_function(fruits)
```

```
apple
banana
cherry
```

1.8 Return Values

To let a function return a value, use the return statement:

[8]:
```
def my_function(x):
  return 5 * x

print(my_function(3))
print(my_function(5))
print(my_function(9))
```

```
15
25
45
```

1.9 The pass Statement

Note: Function definitions cannot be empty, but if you for some reason have a function definition with no content, put in the pass statement to avoid getting an error.

[9]:
```
def myfunction():
  pass
```

[10]:
```
def myfunction():
```

```
  File "<ipython-input-10-fccd18dcbd4e>", line 1
    def myfunction():
                    ^
SyntaxError: unexpected EOF while parsing
```

---

2 Recursion

Python also accepts function recursion, which means a defined function can call itself.

Recursion is a common mathematical and programming concept. It means that a function calls itself. This has the benefit of meaning that you can loop through data to reach a result.

The developer should be very careful with recursion as it can be quite easy to slip into writing a function which never terminates, or one that uses excess amounts of memory or processor power. However, when written correctly recursion can be a very efficient and mathematically-elegant approach to programming.

In this example, tri_recursion() is a function that we have defined to call itself ("recurse"). We use the k variable as the data, which decrements (-1) every time we recurse. The recursion ends when the condition is not greater than 0 (i.e. when it is 0).

To a new developer it can take some time to work out how exactly this works, best way to find out is by testing and modifying it.

[11]:
```
# Recursion Example
def tri_recursion(k):
  if(k > 0):
    result = k + tri_recursion(k - 1)
    print(result)
  else:
    result = 0
  return result

print("\n\nRecursion Example Results")
tri_recursion(6)
```

```
Recursion Example Results
1
3
6
10
```

```
15
21
```

[11]:
```
21
```

2.1 Syntax

```
lambda arguments : expression
```

The expression is executed and the result is returned:

[12]:
```
# A lambda function that adds 10 to the number passed in as an argument, and␣
 ↪print the result:
x = lambda a : a + 10
print(x(5))
```

```
15
```

Lambda functions can take any number of arguments:

[13]:
```
# Example 1 - A lambda function that multiplies argument a with argument b and␣
 ↪print the result:
x = lambda a, b : a * b
print(x(5, 6))

#Output is 30
```

```
30
```

[14]:
```
# Example 2 - A lambda function that sums argument a, b, and c and print the␣
 ↪result:
x = lambda a, b, c : a + b + c
print(x(5, 6, 2))

#Output is 13
```

```
13
```

2.2 Why Use Lambda Functions?

The power of lambda is better shown when you use them as an anonymous function inside another function.

Say you have a function definition that takes one argument, and that argument will be multiplied with an unknown number:

[15]:
```
def myfunc(n):
  return lambda a : a * n
```

Use that function definition to make a function that always doubles the number you send in:

[16]:
```
def myfunc(n):
  return lambda a : a * n

mydoubler = myfunc(2)

print(mydoubler(11))
```

```
22
```

Or, use the same function definition to make a function that always triples the number you send in:

[18]:
```
def myfunc(n):
  return lambda a : a * n

mytripler = myfunc(3)

print(mytripler(11))

#Output is 33
```

```
33
```

Or, use the same function definition to make both functions, in the same program:

[19]:
```
def myfunc(n):
  return lambda a : a * n

mydoubler = myfunc(2)
mytripler = myfunc(3)

print(mydoubler(11))
print(mytripler(11))

#Output is 22
#Output is 33
```

```
22
33
```

2.3 Whitespace

Consider this function greet_customer():

[24]:
```
def greet_customer():
  print("Welcome to Engrossing Grocers.")
  print("Our special is mandarin oranges.")
  print("Have fun shopping!")

greet_customer()
```

```
Welcome to Engrossing Grocers.
Our special is mandarin oranges.
Have fun shopping!
```

The three print statements are all executed together when greet_customer() is called. This is because they have the same level of indentation. In Python, the amount of whitespace tells the computer what is part of a function and what is not part of that function. If we wanted to write another line outside of greet_customer(), we would have to unindent the new line:

[21]:
```
def greet_customer():
  print("Welcome to Engrossing Grocers.")
  print("Our special is mandarin oranges.")
  print("Have fun shopping!")
print("Cleanup on all")
greet_customer()
greet_customer()
```

```
Cleanup on all
Welcome to Engrossing Grocers.
Our special is mandarin oranges.
Have fun shopping!
Welcome to Engrossing Grocers.
Our special is mandarin oranges.
Have fun shopping!
```

When we run this program, the message "Cleanup on all" will be printed once, while the messages in greet_customer() will all be printed twice. This is because we call the function twice, and "Cleanup on all" is not part of the function. Notice also that "Cleanup on all" will be printed before the greet_customer() messages since we call the function after it.

Here we use 2 spaces for our default indentation. Anything other than that will throw an error when you try to run the program. Many other platforms use 4 spaces. Some people even use tabs! These are all fine. What is important is being consistent throughout the project.

Example 1. Passing parametes

The function mult_two_add_three() prints a number multiplied by 2 and added to 3. As it is written right now, the number that it operates on is always 5.

Now, we call the function and see what it prints to the console.

[25]:
```
def mult_two_add_three():##funvtion does not get any parameter
  number = 5
  print(number*2 + 3)

# Call mult_two_add_three() here:
mult_two_add_three()
```

```
13
```

[26]:
```
def mult_two_add_three(number):# here we have to pass one parameter
  #number = 5
  print(number*2 + 3)

# Call mult_two_add_three() here:
mult_two_add_three(1)# Here, we call function and pass one parameter
```

```
5
```

Different type to call mult_two_add_three() function.

[27]:
```
def mult_two_add_three():
  number = 5
  print(number*2 + 3)

# Call mult_two_add_three() here:
mult_two_add_three()

def mult_two_add_three(number):
  #number = 5
  print(number*2 + 3)

# Call mult_two_add_three() here:
mult_two_add_three(1)

def mult_two_add_three(number):
  #number = 5
  print(number*2 + 3)

# Call mult_two_add_three() here:
mult_two_add_three(5)

def mult_two_add_three(number):
  #number = 5
  print(number*2 + 3)

# Call mult_two_add_three() here:
mult_two_add_three(-1)
```

```
def mult_two_add_three(number):
  #number = 5
  print(number*2 + 3)

# Call mult_two_add_three() here:
mult_two_add_three(0)
```

```
13
5
13
1
3
```

2.4 Multiple Parameters

The function **mult_x_add_y()** takes a number, multiplies it by two and adds three. So, we want to make this function more flexible by adding x and y as parameters ot the function after number and replace 2 in the print statement with x, and replace 3 in the print statement with y.

```
def mult_x_add_y(number, x, y):
  print(number*x + y)
```

Test function by calling it and pass parameters: number: 5 x: 2 y: 3

[28]:
```
def mult_x_add_y(number, x, y):
  print(number*x + y)

mult_x_add_y(5, 2, 3)
```

```
13
```

2.5 Returns

So far, we have only seen functions that print out some result to the console. Functions can also return a value to the user so that this value can be modified or used later. When there is a result from a function that can be stored in a variable, it is called a returned function value. We use the keyword return to do this.

Here's an example of a function divide_by_four that takes an integer argument, divides it by four, and returns the result:

[29]:
```
def divide_by_four(input_number):
  return input_number/4
```

```
divide_by_four(4)
```

[29]: 1.0

The program that calls divide_by_four can then use the result later:

[32]:
```
result = divide_by_four(16)
# result now holds 4
print("16 divided by 4 is " + str(result) + "!")
result2 = divide_by_four(result)
print(str(result) + " divided by 4 is " + str(result2) + "!")

#This would print out:
```

```
16 divided by 4 is 4.0!
4.0 divided by 4 is 1.0!
```

2.5.1 Return string

In this example, we returned a number, but we could also return a String:

[33]:
```
def create_special_string(special_item):
  return "Our special is " + special_item + "."

special_string = create_special_string("banana yogurt")

print(special_string)
```

```
Our special is banana yogurt.
```

One more example how to print return values:

[35]:
```
def calculate_age(current_year, birth_year):
  age = current_year - birth_year
  return age

my_age= calculate_age(2049, 1993)

dads_age= calculate_age(2049, 1953)

print("I am X years old and my dad is Y years old" , my_age, dads_age)
```

```
I am X years old and my dad is Y years old 56 96
```

2.6 Multiple Return Values

Sometimes we may want to return more than one value from a function. We can return several values by separating them with a comma:

[36]:
```
def square_point(x_value, y_value):
  x_2 = x_value * x_value
  y_2 = y_value * y_value
  return x_2, y_2
```

This function takes in an x value and a y value, and returns them both, squared. We can get those values by assigning them both to variables when we call the function:

[37]:
```
x_squared, y_squared = square_point(1, 3)
print(x_squared)
print(y_squared)
```

```
1
9
```

[ ]:
```
Example 2:
```

[38]:
```
def get_boundaries(target, margin):
  low_limit = target - margin
  high_limit = margin + target
  return low_limit, high_limit

low, high = get_boundaries(100, 20)
print(low)
print(high)
```

```
80
120
```

2.7 Scope

Let's say we have our function create_special_string() that creates a string about a special item:

[39]:
```
def create_special_string(special_item):
  return "Our special is " + special_item + "."
```

What if we wanted to access the variable special_item outside of the function? Could we use it?

[40]:
```
def create_special_string(special_item):
  return "Our special is " + special_item + "."
```

```
print("I don't like " + special_item)
```

```
      ␣
↪---------------------------------------------------------------------------

       NameError                                 Traceback (most recent call␣
↪last)

       <ipython-input-40-fb8395e058b2> in <module>
         2   return "Our special is " + special_item + "."
         3
   ----> 4 print("I don't like " + special_item)


       NameError: name 'special_item' is not defined
```

If we try to run this code, we will get a NameError, telling us that 'special_item' is not defined. The variable special_item has only been defined inside the space of a function, so it does not exist outside the function. We call the part of a program where special_item can be accessed its scope. The scope of special_item is only the create_special_string function.

Variables defined outside the scope of a function may be accessible inside the body of the function:

[41]:
```
header_string = "Our special is "

def create_special_string(special_item):
  return header_string + special_item + "."
print(create_special_string("grapes"))
```

```
Our special is grapes.
```

There is no error here. header_string can be used inside the create_special_string function because the scope of header_string is the whole file.

[ ]:
```
One more example
```

[43]:
```
current_year = 2048 #global variable

def calculate_age(birth_year): #takes anly one parameter
  age = current_year - birth_year
  return age

print(current_year) #print global variable
age = calculate_age(1970) #this age is different scope than age inside␣
 ↪calculate_age function
print(age) # print age, we save return value
```

```
2048
78
```

2.8 Read user input

In the below program to add two numbers, the user is first asked to enter two numbers and the input is scanned using the input() function and stored in the variables number1 and number2. Then, the variables number1 and number2 are added using the arithmetic operator + and the result is stored in the variable sum.

[45]:
```
# Python3 program to add two numbers

number1 = input("First number: ")
number2 = input("\nSecond number: ")

# Adding two numbers
# User might also enter float numbers
sum = float(number1) + float(number2) # Note! type casting float(number1)...

# Display the sum
# will print value in float
print("The sum of {0} and {1} is {2}" .format(number1, number2, sum)) # Note␣
 ↪{0} to set place where where number1 is printed
```

```
First number:  12

Second number:  32

The sum of 12 and 32 is 44.0
```

2.9 Review

Great! So far you have learned:

```
How to write a function
How to give a function inputs
How to return values from a function
What scope means
```

Let's practice these concepts again so that you won't forget them!

Copy, save your own workspace Chapter 3 - ToDo exercise

[ ]:
```

```